

SWB2 Static Analysis & Code Quality Summary

Prepared for: USGS Code Review

Date: July 2, 2026 (updated)

Author: Steve Westenbroek

Overview

The SWB2 codebase was subjected to comprehensive static analysis using two independent Fortran compilers (GNU gfortran 15.2 and Intel ifx 2025) with maximum warning levels and Fortran 2018 standard conformance checking. All actionable diagnostics have been resolved.

Results

Compiler	Starting Diagnostics	Final Diagnostics	Actionable Remaining
gfortran <code>-std=f2018 -Wall -Wextra -pedantic</code>	540	41	0
ifx <code>/warn:all /stand:f18</code>	487	83	0

gfortran — 41 remaining warnings (all non-actionable)

Category	Count	Explanation
<code>-Wunused-function</code>	20	Intentional reserve API: private functions retained for planned features (NetCDF abstraction, spatial analysis, zone aggregation). Documented in design/unused_functions_analysis.md .
<code>-Wunused-dummy-argument</code>	16	Strategy pattern: procedure pointer interfaces require matching signatures across all implementations; stub/no-op implementations don't use every argument. Structural necessity.
<code>-Wmaybe-uninitialized</code>	5	False positive: allocatable array assigned from derived-type method result. gfortran cannot prove allocation metadata is set, but it always is.

ifx — 83 remaining diagnostics (all informational remarks)

The 83 remaining diagnostics are `remark #7712` (unused dummy argument) — informational notices inherent to Fortran OOP, where type-bound procedure signatures must match across all implementations regardless of whether a specific implementation uses every argument. ifx correctly classifies these as remarks, not warnings.

Issues Fixed (by category)

Category	Count	Impact
Implicit narrowing conversions	89	Precision loss made explicit
Unused variables	289	Dead code removed
Mixed-kind min/max	18	Prevented silent type promotion
Impure function elimination	69	Functions correctly marked <code>pure</code>
Maybe-uninitialized variables	13	3 real bugs found (see below)
Unused private functions (stale)	12	Dead code removed

Category	Count	Impact
Unused module variables/imports	16	Dead declarations removed
Stack-to-static locals	10	Thread-safety corrected
Float equality comparisons	10	Replaced with inequality tests
INQUIRE logical type conformance	16	F2018 standard compliance
CRT linkage conflict (ifx/Windows)	76	Build system corrected
Lines exceeding 132 columns	6	Standard conformance
Argument aliasing	2	Intent corrected
Character truncation	1	Variable width corrected
Total fixed	~640	

Real Bugs Discovered

The static analysis process uncovered three actual correctness bugs:

- solar_calculations.F90 — wrong variable in expression.** The function `solar_altitude__alpha` computed `dAlpha = HALFPI - dAlpha` (using the uninitialized result variable) instead of `dAlpha = HALFPI - dTheta_z` (the input parameter). This function was not called in any active code path, explaining why it went undetected.
- swbstats2_support.F90 — typo in Kahan summation initializer (2 instances).** The compensator variable `sum_val_comp` was never initialized due to a copy-paste typo (`sum_val = 0.0; sum_val = 0.0` instead of `sum_val = 0.0; sum_val_comp = 0.0`). This corrupted zone-based statistical accumulation with garbage initial values.

Design Decisions

- 20 unused functions retained by design:** These form a coherent internal API (NetCDF data access, grid spatial operations, zone aggregation) following a facade pattern. They are private helpers intended for future public entry points. Documented in `design/unused_functions_analysis.md`.
- No initialization at declaration:** Fortran implicitly applies the `SAVE` attribute to variables initialized at declaration, making them persistent between calls (unsafe for reentrancy). All initializations use executable statements instead.
- NODATA comparisons use inequality:** Grid NODATA sentinel values (typically -9999.0) are tested with `<=` rather than `==` to avoid floating-point equality warnings while maintaining correct semantics for the domain.

Build Infrastructure

- Meson build system with three profiles: `release`, `develop`, `static_analysis`
- `static_analysis` profile enables maximum warnings from both compilers
- Normal builds suppress `-Wunused-function` to avoid noise from intentional reserve API
- Windows CRT linkage controlled via `b_vscrt` option (eliminates `/MDd` vs `/MT` conflicts)
- Both gfortran (MSYS2/ucrt64) and ifx (Intel oneAPI) produce clean builds

Supporting Documentation

Document	Contents
<code>design/static_analysis_remediation_plan.md</code>	Detailed tracking of all warning categories and fixes

Document	Contents
design/unused_functions_analysis.md	Analysis of 31 unused functions with disposition rationale
design/maybe_uninitialized_fixes.md	Details on the 13 uninitialized variable fixes including bugs found